Lecture 4.1 – Integral Action and PI Control

Week 4: Removing Steady-State Error in Motor-Speed Control

Learning Objectives

By the end of this lecture, you should be able to:

- explain why P control leaves steady-state error;
- describe what the integral term does;
- write the PI-control law;
- implement the integral term in discrete time;
- explain integral windup;
- describe basic anti-windup protection; and
- connect PI control to the motor-speed project.

1. Review: Why P Control Leaves Error

In Week 3, the proportional controller was

$$u(t) = K_p e(t)$$

where

$$e(t) = r(t) - y(t).$$

Pure P control reduces the error, but it usually does not remove it completely.

At steady state, the motor needs a nonzero PWM command to keep rotating. But if the error became exactly zero,

$$e = 0,$$

then

$$u = K_p e = 0.$$

That would turn the motor command off.

With pure P control, some nonzero steady-state error is usually needed to produce the PWM command required to keep the motor running.

2. The Idea of Integral Action

Integral action adds up error over time.

If the motor is below the desired speed for a long time, then the accumulated error grows.

That accumulated error increases the controller output until the remaining steady-state error becomes smaller.

Mathematically, the integral of error is

$$\int_0^t e(\lambda)\, d\lambda.$$

The integral term remembers past error.

Proportional control reacts to the current error. Integral control reacts to accumulated past error.

3. PI Control Law

A proportional-integral controller is

$$\boxed{u(t) = K_p e(t) + K_i \int_0^t e(\lambda)\, d\lambda}$$

where

- K_p is the proportional gain;
- K_i is the integral gain;
- $e(t)$ is the current error;
- $\int e(t)\, dt$ is accumulated error.

The two terms have different roles:

Term	Meaning	Main effect
$K_p e(t)$	present error	quick correction
$K_i \int e(t)dt$	past accumulated error	reduces steady-state error

4. Why the Integral Term Removes Error

Suppose the desired speed is

$$r = 120 \text{ RPM}$$

and the motor settles at

$$y = 110 \text{ RPM}.$$

Then

$$e = 10 \text{ RPM}.$$

With PI control, this nonzero error keeps adding to the integral term:

$$\int e(t)\, dt \uparrow .$$

The control command increases until the motor speed gets closer to the reference.
As the speed approaches the reference, the error becomes smaller.

> If a constant error remains, the integral term continues to grow and pushes the controller to remove that error.

5. Discrete-Time Integral Implementation

Arduino code runs at sample times, not continuously.
If the sample period is

$$T_s,$$

then the integral can be approximated by a sum:

$$I[k] = I[k-1] + e[k]T_s.$$

The PI controller becomes

$$\boxed{u[k] = K_p e[k] + K_i I[k].}$$

In code:

```
error = referenceRPM - measuredRPM;

integral = integral + error * Ts;

control = Kp * error + Ki * integral;

pwm = constrain(control, 0, 255);
```

6. Units of the Integral Gain

For our motor,

$$e(t)$$

has units of RPM.
The integral

$$\int e(t)\,dt$$

has units of

$$\text{RPM} \cdot \text{s}.$$

Since $u(t)$ is PWM, the integral gain has units

$$\boxed{K_i : \frac{\text{PWM}}{\text{RPM} \cdot \text{s}}.}$$

7. What Happens When K_i Is Too Small or Too Large?

If K_i is too small:

- steady-state error decreases slowly;
- the response may look almost like P control;
- correction after load changes is slow.

If K_i is too large:

- the controller may overcorrect;
- the response may oscillate;
- PWM may stay saturated for too long;
- the motor may sound rough or unstable.

Integral action is powerful, but it must be added carefully. Start with small K_i values.

8. Integral Windup

The PWM command is limited:

$$0 \leq u \leq 255.$$

Suppose the reference speed is high and the motor is far below the target. The controller may request

$$u = 500.$$

But the Arduino can only apply

$$u = 255.$$

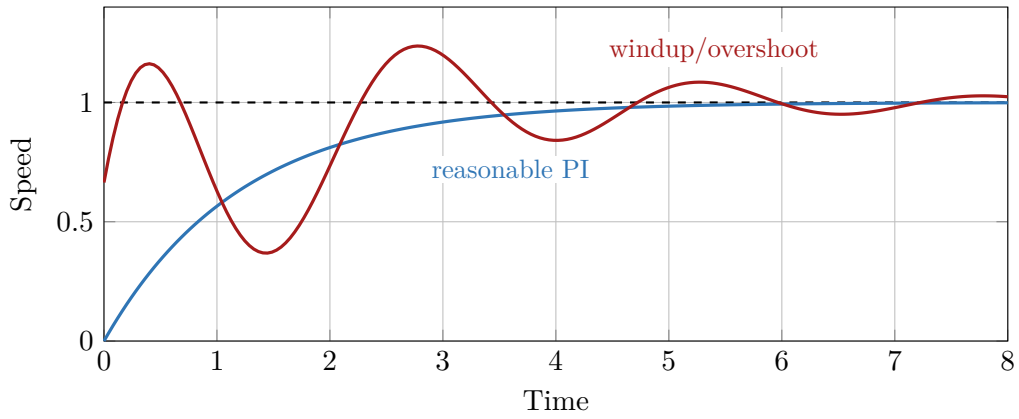
If the error remains large while the output is saturated, the integral term can keep growing. This is called **integral windup**.

Integral windup happens when the integral term keeps accumulating error even though the actuator is already saturated.

9. Why Windup Is a Problem

When the motor finally approaches the target speed, the integral term may be too large. The controller may continue applying too much PWM, causing:

- overshoot;
- long settling time;
- oscillation;
- rough motor behavior.



10. Simple Anti-Windup Idea

A simple anti-windup rule is:

> Do not keep growing the integral term when the PWM command is already saturated in the same direction as the error.

A simpler student-friendly method is integral clamping:

$$I_{\min} \leq I \leq I_{\max}.$$

In code:

```
integral = integral + error * Ts;

integral = constrain(integral, integralMin, integralMax);
```

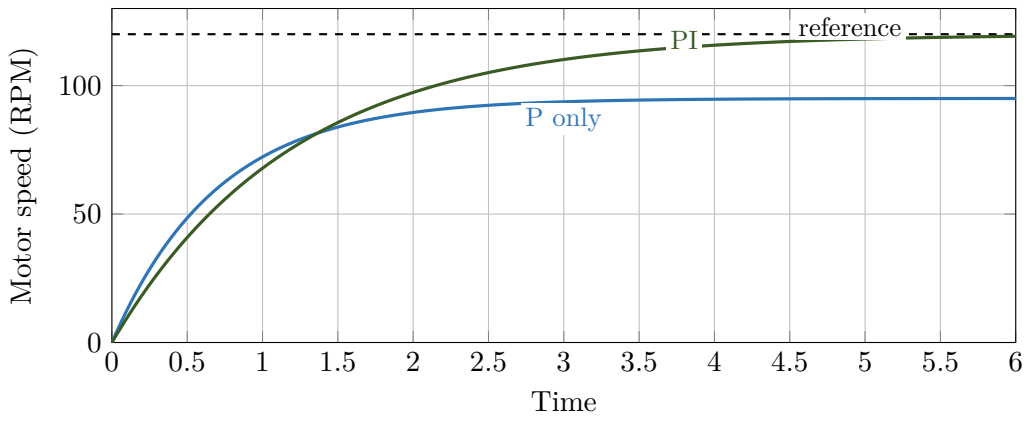
This does not solve every problem, but it is practical and easy to implement.

11. PI Controller Algorithm for the Motor

At every sample time:

1. measure RPM from the encoder;
2. calculate
$$e = r - y;$$
3. update the integral:
$$I = I + eT_s;$$
4. clamp the integral;
5. calculate
$$u = K_p e + K_i I;$$
6. clamp PWM to 0–255;
7. apply PWM to the motor.

12. P vs. PI Behavior



P control can be fast but may leave error. PI control can reduce or remove steady-state error, but tuning becomes more delicate.

13. Practical Tuning Strategy

For this course, use a simple tuning sequence:

1. Start with your selected K_p from Week 3.
2. Set $K_i = 0$.
3. Add a small K_i.
4. Test the response.
5. Increase K_i slowly until steady-state error is acceptably small.
6. Stop increasing K_i if the response overshoots, oscillates, or saturates too long.

Do not tune K_p and K_i randomly at the same time. Change one parameter at a time and record the result.

14. Check Your Understanding

Suppose $r = 120$ RPM, $y = 100$ RPM, $K_p = 2$, $K_i = 0.5$, and the current integral value is $I = 30$ RPM $\cdot$ s.

1. What is the current error?
2. What is the proportional contribution?
3. What is the integral contribution?
4. What is the controller output before PWM clamping?

Summary

- P control uses current error.
- Integral action accumulates past error.
- PI control is
$$u(t) = K_p e(t) + K_i \int e(t)\, dt.$$
- In discrete time,
$$I[k] = I[k-1] + e[k]T_s.$$

- Integral action helps remove steady-state error.
- Integral windup can occur when PWM saturates.
- Anti-windup protection is important in real hardware.